



PRAXISORIENTIERTE FALLSTUDIE (AI3955)

Sommersemester 2026

Verfügbarkeit und Fehlertoleranz des ROSI-Systems

im industriellen Dauerbetrieb

vorgelegt von:

Luca Michael Schmidt (Matr.-Nr. 1540963)

Hochschulbetreuer:

Prof. Dr. Thomas Wiemann

Betreuer im Unternehmen:

Manuel Pilz (Grenzebach BSH GmbH)

Hochschule Fulda – Fachbereich Angewandte Informatik

1. Juni 2026

Sperrvermerk

Die vorliegende Arbeit beinhaltet interne und vertrauliche Informationen der Grenzebach BSH GmbH. Die Weitergabe des Inhalts der Arbeit sowie der darin enthaltenen Daten ist im Ganzen oder in Teilen grundsätzlich untersagt. Es dürfen keine Kopien oder Abschriften, auch nicht in digitaler Form, gefertigt werden. Ausnahmen bedürfen der schriftlichen Genehmigung der Grenzebach BSH GmbH.

Zusammenfassung

Diese praxisorientierte Fallstudie analysiert die Verfügbarkeit und Fehlertoleranz des ROSI-Systems (*Realtime Optical Surface Inspection*) im industriellen 24/7-Betrieb. ROSI ist ein On-Premise-Inline-Scan-System der Grenzebach BSH GmbH zur automatisierten Qualitätskontrolle von Baustoffen und technisch als Python-Multiprocessing-System mit Shared-Memory-basierter Verarbeitungspipeline realisiert.

Der Schwerpunkt liegt auf Fehlern, die im Dauerbetrieb schleichend entstehen, lange unbemerkt bleiben oder kaskadieren und im Entwicklungsbetrieb daher selten sichtbar werden. Fünf Failure Cases auf Prozess- und Betriebssystemebene wurden in einer kontrollierten Testumgebung mittels Fault Injection reproduzierbar untersucht.

Die Messungen belegen erhebliche Lücken in der Fehlertoleranz: Drei von vier Service-Prozessen fallen ohne Erkennung aus (Fail-Silent); eine volle RAM-Disk führt nach einer Stillstandsphase von 35 Sekunden zum Absturz der gesamten Anlage; eine blockierende Log-Queue friert das System für 27 Sekunden ohne Alarm ein. Eine veraltete Datenbankverbindung erweist sich als produktionsspezifisches Risiko, das durch die Entwicklungskonfiguration maskiert wird. Eine zunächst angenommene Race Condition im Speichermanagement konnte hingegen als durch Timing-Reserven und idempotente Freigabe abgesichert widerlegt werden. Auf Basis der Befunde werden für jeden Case konkrete, aufwandsarme Handlungsempfehlungen abgeleitet.

Inhaltsverzeichnis

1	Einleitung und Problemstellung	1
1.1	Problemstellung	1
1.2	Zielsetzung und Forschungsfragen	2
1.3	Aufbau der Arbeit	2
2	Theoretische Grundlagen	2
2.1	Verfügbarkeit im Dauerbetrieb	2
2.2	Fehlerklassifikation	3
2.3	Prozesssupervision und das Watchdog-Pattern	3
2.4	Ressourcenerschöpfung und Backpressure	4
2.5	Dev/Prod-Parity und Konfigurationsgefälle	4
3	Methodik	5
3.1	Untersuchungsansatz	5
3.2	Testumgebung	5
3.3	Referenzmessung (Baseline)	5
3.4	Messgrößen	5
4	Failure Cases	6
4.1	Case 1 – Lücken in der Laufzeit-Prozesssupervision	6
4.2	Case 2 – Flüchtiger Bildspeicher als Single Point of Failure	7
4.3	Case 3 – Log-Queue-Stau als Pipeline-Blocker	8
4.4	Case 4 – Veraltete Datenbankverbindung nach Produktionspause	9
4.5	Case 5 – Vermutete Race Condition im Speichermanagement	9
5	Fazit und Handlungsempfehlungen	10
5.1	Übergreifende Befunde	10
5.2	Priorisierte Handlungsempfehlungen	11
5.3	Ausblick	11
	Literaturverzeichnis	12

1 Einleitung und Problemstellung

ROSI (*Realtime Optical Surface Inspection*) ist ein von der Grenzebach BSH GmbH entwickeltes Inline-Scan-System zur automatisierten Qualitätskontrolle von Baustoffen. Im laufenden Produktionsprozess werden Materialoberflächen erfasst, Defekte per KI-Inferenz erkannt und das Material anschließend regelbasiert in Qualitätsstufen klassifiziert. Der Betrieb erfolgt On-Premise direkt in der Produktionsanlage des Kunden im 24/7-Dauerbetrieb.

Technisch ist ROSI als Python-Multiprocessing-System aufgebaut. Mehrere langlebige Prozesse verarbeiten die Daten in einer Shared-Memory-basierten Pipeline, ergänzt durch Queues und flüchtigen Zwischenspeicher in Form eines tmpfs-RAM-Dateisystems. Große Bilddaten wandern dabei nicht durch Queues, sondern verbleiben in vom Betriebssystem verwalteten Speicherblöcken; die Queues transportieren ausschließlich Referenzen. Die in dieser Fallstudie untersuchten Fehlerklassen setzen genau an dieser Prozess- und Speicherarchitektur an.

ROSI befindet sich in aktiver Entwicklung; die erste Kundeninbetriebnahme ist für Mitte bis Ende 2026 geplant. Mit dem bevorstehenden Produktiveinsatz steigen die Anforderungen an Verfügbarkeit und Fehlertoleranz erheblich: Ausfälle können Produktionsstillstände verursachen, Inspektionsergebnisse verhindern und im ungünstigsten Fall Qualitätsmängel unentdeckt lassen.

1.1 Problemstellung

Systeme im 24/7-Betrieb sind Fehlerklassen ausgesetzt, die im Entwicklungsbetrieb unsichtbar bleiben. Ein System, das täglich neu gestartet wird und unter manueller Beobachtung steht, durchläuft selten jene Zustände, in denen sich Ressourcen schleichend erschöpfen, Verbindungen veralten oder Teilprozesse still ausfallen. Für ROSI lassen sich die relevanten Risikokategorien wie folgt einteilen:

- *Speichererschöpfung*: Puffer, Logs oder Datenbanken wachsen schleichend, bis Schreiboperationen systemweit fehlschlagen.
- *Stille Prozessausfälle*: Ein Teilprozess stürzt ab, das System läuft scheinbar weiter – ohne Alarm und ohne Neustart.
- *Verbindungsabbrüche*: Datenbankverbindungen veralten nach Produktionspausen und führen beim nächsten Zugriff zu unkontrollierten Prozessabstürzen.
- *Kaskadeneffekte*: Die Verlangsamung einer Komponente blockiert über gemeinsame Queues und Shared-Memory-Schnittstellen das gesamte System.
- *Blinde Flecken im Monitoring*: Vorhandene Überwachungskomponenten erfassen nicht alle kritischen Ressourcen, etwa flüchtige RAM-Dateisysteme oder interne Queue-Tiefen.

ROSI verfügt aktuell über kein externes Alerting. Probleme werden erst durch ausbleibende Inspektionsergebnisse oder manuelle Beobachtung sichtbar.

1.2 Zielsetzung und Forschungsfragen

Ziel dieser Fallstudie ist es, sicherheitskritische Fehlerszenarien im industriellen Dauerbetrieb auf Prozess- und Betriebssystemebene zu identifizieren, reproduzierbar zu simulieren und anhand messbarer Kriterien systematisch zu bewerten. Im Fokus stehen stille, schleichende und kaskadierende Fehler. Die untersuchten Failure Cases werden in drei Themenblöcke gegliedert, denen jeweils eine Forschungsfrage (FF) zugeordnet ist:

- *FF1 – Speicher- und Ressourcenverwaltung:* Wie verhält sich das System, wenn flüchtige Speichersubsysteme überlaufen oder Timing-Konflikte im Shared-Memory-Management auftreten?
- *FF2 – Systembeobachtbarkeit und Fehlertoleranz:* In welchem Zeitraum erkennt das System eigene Prozessausfälle, und welche Reaktion erfolgt?
- *FF3 – Datenpersistenz und Kaskadeneffekte:* Wie verhält sich das System bei unterbrochenen Datenbankverbindungen oder blockierenden Queues, und welche Folgeausfälle entstehen?

1.3 Aufbau der Arbeit

Kapitel 2 legt den theoretischen Rahmen zu Verfügbarkeit, Fehlerklassifikation, Prozesssupervision und Ressourcenmanagement. Kapitel 3 beschreibt die Testumgebung und die Methodik der Fehlerinjektion. Kapitel 4 stellt die fünf Failure Cases mit ihren Messergebnissen dar. Kapitel 5 fasst die Befunde zusammen und leitet Handlungsempfehlungen ab.

2 Theoretische Grundlagen

2.1 Verfügbarkeit im Dauerbetrieb

Verfügbarkeit (*Availability*) ist eine zentrale Eigenschaft zuverlässiger Systeme. Avizienis et al. definieren sie als die Bereitschaft eines Systems, seinen spezifizierten Dienst korrekt zu erbringen [1]. Formal ergibt sie sich aus der mittleren Betriebsdauer zwischen Ausfällen (*Mean Time to Failure*, MTTF) und der mittleren Wiederherstellungszeit (*Mean Time to Repair*, MTTR):

$$A = \frac{MTTF}{MTTF + MTTR} \quad (1)$$

Gleichung 1 verdeutlicht, dass hohe Verfügbarkeit nicht allein durch seltene Ausfälle, sondern ebenso durch kurze Wiederherstellungszeiten erreicht wird. Für ein System ohne automatische Fehlererkennung ist die MTTR untrennbar an die Zeit gebunden, bis ein Mensch den Ausfall bemerkt. Während Entwicklungsumgebungen typischerweise täglich neu gestartet werden und Fehler durch manuelle Beobachtung sichtbar sind, akkumulieren sich im Dauerbetrieb Effekte, die über kurze Beobachtungsfenster unsichtbar bleiben. Ein System, das im Entwicklungsbetrieb stabil läuft, kann daher im 24/7-Betrieb durch eine andere Klasse von Fehlern ausfallen.

2.2 Fehlerklassifikation

Cristian unterscheidet grundlegende Verhaltensklassen bei Prozessausfällen in verteilten Systemen [2]. Für diese Fallstudie sind zwei Klassen relevant. Bei einem *Fail-Stop*-Ausfall bricht ein fehlerhafter Prozess sofort ab und signalisiert seinen Ausfall unmissverständlich; der Fehler ist für das Gesamtsystem sichtbar und adressierbar. Bei einem *Fail-Silent*-Ausfall stellt der Prozess seine Funktion ohne Meldung ein, das Gesamtsystem läuft scheinbar weiter – ohne Alarm, ohne Neustart, ohne Log. Fail-Silent-Verhalten ist in industriellen Systemen besonders gefährlich, weil der Ausfall nicht durch technische Signale, sondern erst durch ausbleibende Produktionsergebnisse sichtbar wird.

Davon abzugrenzen sind *schleichende Fehler*, die nicht durch ein singuläres Ereignis, sondern durch die kontinuierliche Akkumulation eines Ressourcenverbrauchs entstehen. Charakteristisch ist die erhebliche Zeitspanne zwischen Ursache und sichtbarem Symptom; im 24/7-Betrieb kann diese Stunden oder ganze Schichten betragen. Birman beschreibt diesen Zustand als nominal laufendes System, das seinen eigentlichen Zweck nicht mehr erfüllt [3].

Ein *Kaskadeneffekt* liegt vor, wenn der Ausfall oder die Verlangsamung einer Komponente über gemeinsame Schnittstellen den Ausfall weiterer Komponenten verursacht [4]. Solche Effekte sind schwer zu diagnostizieren, da das sichtbare Symptom nicht die Ursache widerspiegelt: Das System kollabiert an einer anderen Stelle, als die ursprüngliche Schwachstelle liegt.

2.3 Prozesssupervision und das Watchdog-Pattern

In langlebigen Multiprocessing-Systemen ist es unzureichend, Prozesse nur beim Start zu überprüfen. Das *Supervisor-Modell* sieht einen dedizierten Überwachungsprozess vor, der den Laufzeitstatus aller untergeordneten Prozesse kontinuierlich beobachtet und bei Ausfall automatisch reagiert – durch Neustart, Alarm oder geordnetes Herunterfahren. Dieses Konzept ist im Erlang/OTP-Framework als *Supervision Tree* etabliert, in dem Supervisoren ihre Kindprozesse überwachen und bei Absturz neu starten [5]. Das *Watchdog-Pattern* ist eine konkrete Ausprägung: Der Supervisor erwartet in regelmäßigen Abständen ein Lebenszeichen (*Heartbeat*) des überwachten Prozesses; bleibt es aus, gilt der Prozess als ausgefallen. Für eingebettete

und industrielle Systeme beschreiben Pont und Ong mehrere Varianten dieses Musters mit unterschiedlichen Reaktionsstrategien [6].

Fehlt eine solche Logik, entstehen zwei kritische Szenarien. Im ersten hält ein abgestürzter Prozess, dessen Ausgabe von anderen Prozessen erwartet wird, das System in einem wartenden Zustand gefangen – ohne Timeout, ohne Alarm. Im zweiten läuft das System nach dem Ausfall eines Prozesses scheinbar weiter, Fehler werden erst durch ausbleibende Ausgaben sichtbar. Beide Szenarien erfordern manuelle Eingriffe, die im 24/7-Betrieb ohne Bereitschaftsdienst nicht zeitnah erfolgen.

2.4 Ressourcenerschöpfung und Backpressure

Eine *unbounded Queue* wächst ohne Obergrenze; bei anhaltender Produktionsrate und gedrossem Konsum akkumuliert sie Einträge, bis der Speicher erschöpft ist. Eine *bounded Queue* begrenzt das Wachstum, erzeugt dafür aber *Backpressure*: Produzenten, die in eine volle Queue schreiben wollen, werden blockiert. Kleppmann beschreibt Backpressure als notwendigen, aber häufig fehlerhaft implementierten Mechanismus – ein blockierendes `put()` ohne Timeout überträgt den Stau transitiv auf alle vorgelagerten Komponenten [7]. In einem Multiprocessing-System mit gemeinsamer Queue kann so ein einzelner langsamer Konsument das gesamte System zum Einfrieren bringen.

Flüchtige Dateisysteme wie `tmpfs` und RAM-Disks liegen ausschließlich im Arbeitsspeicher [8]. Sie bieten hohe I/O-Geschwindigkeit, sind jedoch auf eine feste Größe begrenzt: Überschreitet das Schreibvolumen diese Grenze, schlagen Schreiboperationen mit `OSError: No space left on device` fehl. Eine RAM-Disk, die als Puffer zwischen schnellem Schreiben und langsamem Lesen dient, ist strukturell ein *Single Point of Failure* für alle abhängigen Komponenten.

2.5 Dev/Prod-Parity und Konfigurationsgefälle

Das Prinzip der *Dev/Prod-Parity* (Faktor X des *Twelve-Factor App*-Modells) fordert, Entwicklungs- und Produktionsumgebung so ähnlich wie möglich zu halten [9]. Abweichungen in Konfigurationsparametern können dazu führen, dass Fehler in der Entwicklung nicht reproduzierbar sind, die in Produktion deterministisch auftreten. Ein typisches Muster ist das Konfigurationsgefälle beim Verbindungs-Pooling: Entwicklungsumgebungen öffnen pro Anfrage eine frische Verbindung, Produktionsumgebungen nutzen aus Performance-Gründen persistente Verbindungen mit langer Lebensdauer. Die Fehlerklasse der veralteten Verbindung (*stale connection*) tritt damit im Entwicklungsbetrieb strukturell nicht auf. Fehlerbehandlungscode kann fehlen, ohne dass Tests darauf hinweisen; in Produktion führt das erste Auftreten der Fehlerbedingung zum unkontrollierten Absturz.

3 Methodik

3.1 Untersuchungsansatz

Die Untersuchung folgt dem Prinzip der *Fault Injection*: dem gezielten Einbringen definierter Fehlerzustände in ein laufendes System, um dessen Reaktion empirisch zu beobachten [10]. Für jeden Failure Case wird eine Hypothese über das erwartete Systemverhalten formuliert, ein reproduzierbares Fehlerszenario hergestellt und das tatsächliche Verhalten anhand definierter Messgrößen erfasst. Wo ein Fehlerzustand ohne Eingriff in den Quellcode herstellbar ist, werden ausschließlich POSIX-Signale oder externe Werkzeuge eingesetzt; wo ein Eingriff nötig ist, wird dieser minimal gehalten und nach der Messung automatisch zurückgenommen.

3.2 Testumgebung

Alle Messungen erfolgen am Referenzsystem im Labor Bad Hersfeld. Das System wird über einen Simulator betrieben, der die Kamerahardware durch das Einspielen vorab erfasster Bilddaten ersetzt. Sämtliche Failure Cases sind dadurch ohne echte Kamerahardware unter kontrollierten Bedingungen reproduzierbar. Tabelle 1 fasst die relevanten Eigenschaften zusammen.

Tabelle 1: Eigenschaften der Testumgebung

Eigenschaft	Wert
Betriebssystem	Linux 6.17 (x86_64)
Arbeitsspeicher	64 GB
GPU	NVIDIA GeForce RTX 5090 (32 GB VRAM)
Laufzeitumgebung	Python 3.13
Startskript	<code>main.py</code> (mit geänderter Konfiguration für Simulation)
Bilddatenquelle	462 Bilder = 154 Inspektionen

3.3 Referenzmessung (Baseline)

Vor der Fehlerinjektion wurde eine Referenzmessung im Normalbetrieb ohne injizierte Fehler durchgeführt. Sie dient als Vergleichsbasis für alle Failure Cases. Das System startet neben dem Hauptprozess vier Service-Prozesse für `Logger`, `FrameGrabber`, `InspectionDataHandler` und `PipelineManager` sowie einen Pool von 12 Worker-Prozessen. Im stationären Zustand erreicht es einen Durchsatz von 1,74 Inspektionen pro Sekunde bei einer mittleren Pipeline-Laufzeit von 383 ms. Die zentrale Log-Queue bleibt dabei nahezu leer.

3.4 Messgrößen

Für jeden Case werden vergleichbare Messgrößen erhoben:

- *Time-to-Detection (TTD)*: Zeit von der Fehlerinjektion bis zur Erkennung durch das System (Shutdown oder Statusmeldung).
- *Folgeverhalten*: qualitative Beschreibung der Reaktion der übrigen Prozesse (Weiterlauf, Deadlock, Leerschleife, Absturz).
- *Queue-Tiefe und Speicherfüllstand über die Zeit*: quantitative Erfassung schleichender und kaskadierender Effekte.
- *Graceful Degradation*: ob das System bei einem Teilausfall in einen kontrollierten Zustand übergeht oder vollständig ausfällt.

4 Failure Cases

Dieses Kapitel stellt die fünf untersuchten Failure Cases dar. Jeder Case erläutert den relevanten Systemkontext und die zugrunde liegende Schwachstelle, beschreibt das gemessene Verhalten und leitet daraus Interpretation und Handlungsempfehlung ab.

4.1 Case 1 – Lücken in der Laufzeit-Prozesssupervision

Im stationären Betrieb überwacht das System ausschließlich den `PipelineManager` über einen Poll im Sekundentakt. Die drei weiteren Service-Prozesse für Frame-Grabbing, Datenbereitstellung und Logging laufen nach erfolgreichem Start ohne jede Laufzeitüberwachung; eine Supervisor-Logik im Sinne von Abschnitt 2.3 existiert nicht. Um die Auswirkung dieser Lücke zu quantifizieren, wurde jeder Service-Prozess einzeln per `SIGKILL` zum Absturz gebracht und die Time-to-Detection erfasst (Tabelle 2).

Tabelle 2: Reaktion auf den gezielten Ausfall einzelner Service-Prozesse

Zielprozess	Detektiert	TTD	Verhalten
PipelineManager	ja	0,992 s	geordneter Shutdown aller Subsysteme
FrameGrabber	nein	–	Deadlock in Pipeline-Schritt 1
InspDataHandler	nein	–	stiller Ausfall, laufende Inspektionen enden
Logger	nein	–	stiller Ausfall, kein Log-Output mehr

Lediglich der Absturz des `PipelineManagers` wird erkannt, und zwar nach rund einer Sekunde, was dem Poll-Intervall entspricht. Alle übrigen Ausfälle bleiben unbemerkt und entsprechen dem Fail-Silent-Verhalten nach Cristian [2]. Besonders kritisch ist der Ausfall des `FrameGrabbers`: Die Pipeline wartet unbegrenzt auf das nächste Bild, die Anlage steht still, ohne dass ein Alarm ausgelöst wird. Abhilfe schafft eine Erweiterung der Überwachungsschleife auf alle Service-Prozesse oder die Auslagerung der Supervision an eine `systemd`-Unit mit `Restart=on-failure`.

4.2 Case 2 – Flüchtiger Bildspeicher als Single Point of Failure

Verarbeitete Bilder werden auf einer als tmpfs-RAM-Disk gemounteten Partition abgelegt. Die zuständige Funktion `store_images_to_disk` enthält keine Fehlerbehandlung für `OSError`, und der Füllstand wird nicht überwacht. Da eine RAM-Disk auf eine feste Größe begrenzt ist (Abschnitt 2.4), schlagen Schreiboperationen bei Erschöpfung fehl. Im Normalbetrieb wächst das Bildverzeichnis um rund 21,8 MB pro Inspektion, was bei Produktionsdurchsatz einer Schreibrate von etwa 38 MB/s entspricht. Bei 12 GB freiem Speicher wäre die RAM-Disk damit nach ungefähr fünf Minuten erschöpft, sofern keine Rotation oder Auslagerung greift. Abbildung 1 zeigt den gemessenen Anstieg des Bildverzeichnisses im Referenzlauf. Zur Analyse des Fehlerfalls wurde eine volle Disk durch das gezielte Auslösen eines `OSError(28)` nachgebildet (Tabelle 3).

Tabelle 3: Verhalten bei erschöpftem Bildspeicher

Messgröße	Wert
Schreibrate (Hochrechnung Produktionslast)	≈ 38 MB/s
Zeit von <code>OSError</code> bis PM-Absturz	35,2 s
PM-Absturz bis Shutdown	2,1 s
Graceful Degradation	nein

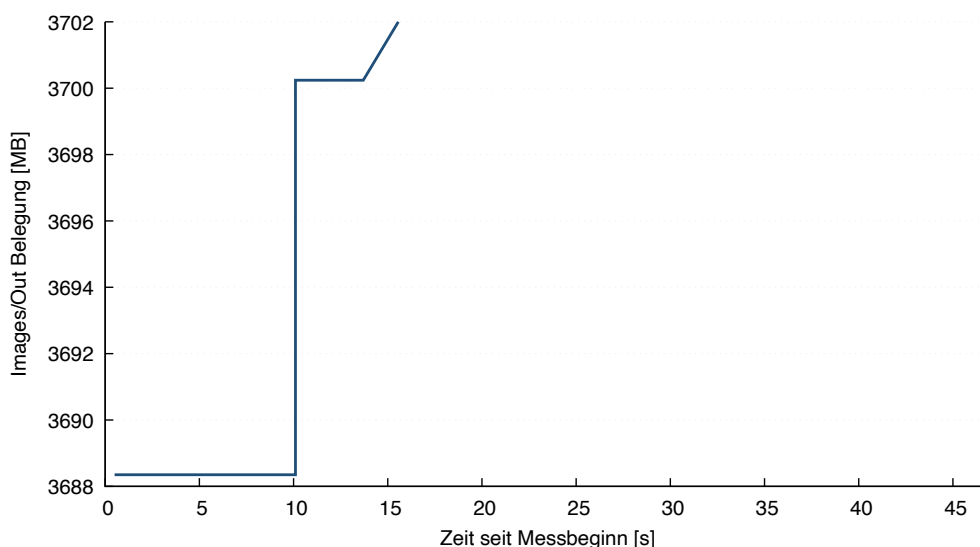


Abbildung 1: Belegung des Bildverzeichnisses `Images/Out` über die Messdauer. In der Entwicklungsumgebung liegt dieser Pfad auf einem regulären Dateisystem; in der Produktionskonfiguration ist er als tmpfs-RAM-Disk vorgesehen.

Quelle: Eigene Darstellung auf Basis eigener Messdaten

Ein einzelner fehlgeschlagener Schreibvorgang fährt das gesamte System herunter. Zwischen dem Fehler und dem erkannten Absturz liegt eine Stillstandsphase von gut 35 Sekunden, in der der PipelineManager vergeblich auf das Ergebnis des abgestürzten Workers wartet. Die RAM-Disk ist damit ein Single Point of Failure ohne Rückfallebene. Wichtig ist die Abgren-

zung zu `/dev/shm`: Shared-Memory-Blöcke und Bildarchiv liegen auf unterschiedlichen Pfaden, teilen sich aber denselben physischen Arbeitsspeicher. Eine Fehlerbehandlung in `store_images_to_disk`, die die Inspektion fortsetzt und lediglich die Bildarchivierung überspringt, sowie ein Füllstands-Monitoring mit Alarm unterhalb von 20 % freiem Speicher würden diesen Ausfall verhindern.

4.3 Case 3 – Log-Queue-Stau als Pipeline-Blocker

Alle Worker und der PipelineManager schreiben Log-Einträge synchron über `log_queue.put()` in eine zentrale, auf 1000 Einträge begrenzte Queue. Der Aufruf erfolgt blockierend ohne Timeout, und der Zustand `queue.Full` wird nicht abgefangen – eine Konstruktion, die Backpressure im Sinne von Abschnitt 2.4 erzeugt. Um eine hängende Datenbankschicht nachzubilden, wurde der Logger per SIGSTOP eingefroren (Tabelle 4).

Tabelle 4: Verhalten bei eingefrorenem Logger-Prozess

Messgröße	Wert
Durchsatz vor Eingriff	17 Pipelines / 5 s
Pipelines nach SIGSTOP	5 (bereits in Bearbeitung)
Zeit bis vollständiger Freeze	$\approx 1,3$ s
Freeze-Dauer (Beobachtung)	27,7 s, kein Alarm
Drain nach SIGCONT	$< 0,3$ s

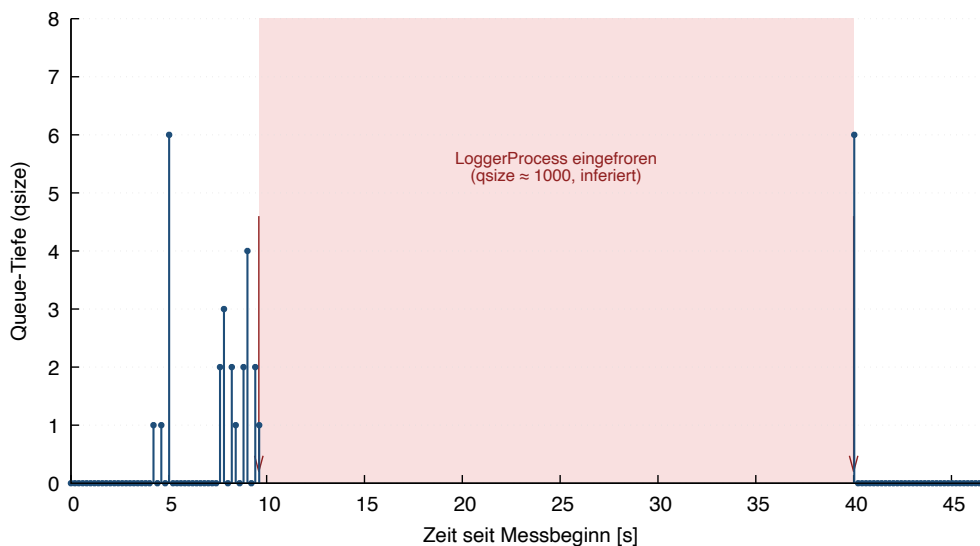


Abbildung 2: Queue-Tiefe der Log-Queue über die Zeit. Im Normalbetrieb bleibt sie nahe null. Während des per SIGSTOP erzwungenen Logger-Freeze (schattiert) pausiert der Mess-Thread mit, sodass keine Messpunkte vorliegen; die Queue läuft inferiert auf ihr Maximum von 1000. Nach SIGCONT bei 40 s drainiert sie in unter 0,3 s.

Quelle: Eigene Darstellung auf Basis eigener Messdaten

Wie Abbildung 2 zeigt, bleibt die Queue im Normalbetrieb nahezu leer. Sobald sie gefüllt

ist, blockieren sämtliche schreibenden Prozesse beim nächsten Log-Aufruf, und die gesamte Pipeline friert ein – eine eigentlich nebensächliche Komponente wirkt so als systemweites Backpressure-Ventil. In der Messung drainiert die Queue nach Freigabe in unter 0,3 Sekunden; der Engpass liegt damit nicht in der normalen Schreibgeschwindigkeit des Loggers, sondern in der fehlenden Entkopplung beim Blockadefall. Es handelt sich um einen Kaskadeneffekt nach [4]: Das Symptom (Pipeline-Freeze) liegt fern der Ursache (blockierende Log-Queue). Eine Umstellung auf `put ( timeout=0 . 1 )` mit Drop-on-Full hält das System im Stau stabil, indem einzelne Log-Einträge verworfen werden.

4.4 Case 4 – Veralterte Datenbankverbindung nach Produktionspause

Die Persistierung der Inspektionsergebnisse erfolgt über das Django-ORM in der Funktion `convert_inspection_to_model`, die weder einen `OperationalError` abfängt noch `close_old_connections()` aufruft. Entscheidend ist ein Konfigurationsgefälle (Abschnitt 2.5): Während die Entwicklung mit `CONN_MAX_AGE = 0` pro Aufruf eine frische Verbindung öffnet, hält die Produktion mit `CONN_MAX_AGE = 60` eine persistente Verbindung. Zur Prüfung wurden in der Entwicklungsumgebung während des Betriebs alle Datenbankverbindungen über `pg_terminate_backend` getrennt. Vor dem Eingriff waren rund 27 Inspektionen verarbeitet; 27 Datenbankverbindungen wurden terminiert. Das System verarbeitete unverändert weiter, ein `OperationalError` trat nicht auf. Dieses Ergebnis ist erwartungskonform, da ohne persistente Verbindung keine veralten kann – das Experiment bestätigt damit gerade, weshalb die Schwachstelle im Labor unsichtbar bleibt.

Das Risiko ist produktionsspezifisch. Nach einer Pause trennt PostgreSQL oder eine vorgelegte Firewall die inaktive Verbindung; der erste Schreibzugriff danach trifft auf eine für gültig gehaltene, tatsächlich tote Verbindung und löst einen `OperationalError` aus, der mangels Behandlung zum Systemabsturz führt. Der Fehler tritt damit deterministisch beim ersten Materialdurchlauf nach jeder Produktionspause auf und wird durch die Entwicklungskonfiguration vollständig maskiert. Ein Aufruf von `close_old_connections()` vor dem Schreibzugriff oder das Setzen von `CONN_MAX_AGE = 0` auch in Produktion behebt das Problem; die zusätzliche Latenz ist bei lokaler Datenbank vernachlässigbar [11].

4.5 Case 5 – Vermutete Race Condition im Speichermanagement

Worker-Prozesse geben ihre Shared-Memory-Blöcke über eine Routine frei, die Blöcke nach mehr als zehn Sekunden ohne Zugriff zerstört [12]. Die Ausgangshypothese lautete, dass diese Freigabe unter Last Blöcke zerstören könnte, die der `PipelineManager` noch benötigt, und so einen `FileNotFoundError` auslöst. Um den vermuteten Konflikt zu provozieren, wurde die Freigabeschwelle von zehn auf zwei Sekunden gesenkt und ein nachgelagerter

Verarbeitungsschritt künstlich um vier Sekunden verzögert. Über 120 Sekunden Laufzeit trat jedoch weder ein `FileNotFoundException` noch ein `BrokenPipeError` oder ein Absturz auf.

Die Hypothese wird durch zwei unabhängige Mechanismen widerlegt. Erstens greift der letzte lesende Schritt typisch nach 45 bis 107 Millisekunden auf die Blöcke zu, während die Freigabe frühestens nach zehn Sekunden erfolgt – ein Sicherheitspuffer von rund dem Hundertfachen. Zweitens ist die Freigabeoperation idempotent und gibt bei einem zweiten Aufruf lediglich einen negativen Status zurück, statt abzustürzen. Der zunächst vermutete kritische Pfad über einen langsamen Persistierungsschritt entfällt zudem, da dieser nicht auf die Speicherblöcke zugreift. Damit besteht kein unmittelbarer Handlungsbedarf; der Befund unterstreicht vielmehr den Wert der empirischen Prüfung, da sich eine plausibel erscheinende Schwachstelle als hinreichend abgesichert erweist. Zur defensiven Absicherung kämen langfristig ein Referenzzähler für Speicherblöcke sowie ein Monitoring der Laufzeit des lesenden Schritts in Betracht.

5 Fazit und Handlungsempfehlungen

Die Fallstudie hat fünf Failure Cases des ROSI-Systems auf Prozess- und Betriebssystemebene untersucht und damit die drei eingangs formulierten Forschungsfragen beantwortet.

Zu FF2 (Systembeobachtbarkeit) zeigt Case 1 die gravierendste Lücke: Drei von vier Service-Prozessen fallen ohne Erkennung aus. Da das System über kein externes Alerting verfügt, bleibt die Time-to-Detection in diesen Fällen unbegrenzt, was sich unmittelbar negativ auf die Wiederherstellungszeit und damit die Verfügbarkeit nach Gleichung 1 auswirkt. Zu FF1 (Ressourcenverwaltung) belegt Case 2 die fehlende Rückfallebene des flüchtigen Bildspeichers, während Case 5 verdeutlicht, dass nicht jede plausible Schwachstelle auch real ist. Zu FF3 (Datenpersistenz und Kaskaden) zeigen Case 3 und Case 4 zwei unterschiedliche Mechanismen: einen systemweiten Freeze durch Backpressure sowie ein durch die Entwicklungskonfiguration maskiertes, produktionsspezifisches Verbindungsrisiko.

5.1 Übergreifende Befunde

Zwei Muster ziehen sich durch die Ergebnisse. Erstens fehlt durchgängig eine *Graceful Degradation*: Ein lokaler Fehler – ein abgestürzter Prozess, ein fehlgeschlagener Schreibvorgang, eine tote Verbindung – führt regelmäßig zum Ausfall des Gesamtsystems statt zu einem kontrollierten Teilbetrieb. Zweitens bestehen *blinde Flecken im Monitoring*: Weder Queue-Tiefen noch Speicherfüllstände noch der Zustand der Service-Prozesse werden zur Laufzeit überwacht. Die Cases 3, 4 und 5 stehen zudem in einem Kaskadenzusammenhang, da eine verlangsamte Datenbank über die Log-Queue das gesamte System ausbremsen und so weitere Fehlerzustände

begünstigen kann.

5.2 Priorisierte Handlungsempfehlungen

Tabelle 5 fasst die abgeleiteten Maßnahmen zusammen. Die ersten vier Maßnahmen greifen direkt in die Verfügbarkeit ein, weil sie entweder die Time-to-Detection verkürzen oder einen lokalen Fehler vom Gesamtsystem entkoppeln. Case 5 erfordert dagegen keinen Sofortfix, sollte aber als Monitoring-Indikator für langsame Bildspeicherung genutzt werden.

Tabelle 5: Priorisierte Handlungsempfehlungen

Case	Aufwand	Maßnahme
1	gering	Supervision aller Service-Prozesse oder <code>systemd</code> -Unit
2	gering	Fehlerbehandlung in der Bildspeicherung + Füllstands-Monitoring
3	minimal	<code>put(timeout)</code> mit Drop-on-Full in der Log-Queue
4	minimal	<code>close_old_connections()</code> bzw. <code>CONN_MAX_AGE = 0</code>
5	–	kein Sofortfix; optional Referenzzähler und Step-5-Monitoring

5.3 Ausblick

Die größten Hebel für die Produktionsreife liegen in einer durchgängigen Laufzeit-Supervision aller Prozesse sowie einem Monitoring der bislang unbeobachteten Ressourcen. Beide Maßnahmen adressieren nicht nur einzelne Cases, sondern die übergreifenden Muster fehlender Beobachtbarkeit und fehlender Graceful Degradation. Eine Übertragung der Untersuchung auf das spätere Kundensystem – mit abweichender Hardware und aktiver Produktionskonfiguration – bleibt offen und würde insbesondere das in Case 4 analytisch hergeleitete Verbindungsrisiko empirisch absichern.

Literaturverzeichnis

- [1] A. Avizienis, J.-C. Laprie, B. Randell und C. Landwehr, „Basic Concepts and Taxonomy of Dependable and Secure Computing“, *IEEE Transactions on Dependable and Secure Computing*, Jg. 1, Nr. 1, S. 11–33, 2004. DOI: 10.1109/TDSC.2004.2
- [2] F. Cristian, „Understanding fault-tolerant distributed systems“, *Communications of the ACM*, Jg. 34, Nr. 2, S. 56–78, 1991. DOI: 10.1145/102792.102801
- [3] K. P. Birman, *Guide to Reliable Distributed Systems: Building High-Assurance Applications and Cloud-Hosted Services*. Springer, 2012, ISBN: 978-1-4471-2415-3.
- [4] H. Qi, S. Bhatt und A. Bhatt, „System Reliability Under Cascading Failure Models“, *IEEE Transactions on Reliability*, 2016. DOI: 10.1109/TR.2015.2500283
- [5] J. Armstrong, *Programming Erlang: Software for a Concurrent World*. Pragmatic Bookshelf, 2007, ISBN: 978-1-934356-00-5.
- [6] M. J. Pont und R. H. L. Ong, *Using watchdog timers to improve the reliability of TTCS embedded systems: Seven new patterns and a case study*, VikingPLoP 2002. [Online]. Available: <https://www.cs.le.ac.uk/SoftSD/resources/viking02.pdf>, Zuletzt abgerufen: 1. Juni 2026, 2002.
- [7] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media, 2017, ISBN: 978-1-4493-7332-0.
- [8] The Linux Kernel Organization, *tmpfs – a virtual memory filesystem*, Linux Kernel Documentation. [Online]. Available: <https://www.kernel.org/doc/html/latest/filesystems/tmpfs.html>, Zuletzt abgerufen: 30. Mai 2026.
- [9] A. Wiggins, *The Twelve-Factor App*, [Online]. Available: <https://www.12factor.net/>, Zuletzt abgerufen: 1. Juni 2026, 2011.
- [10] M.-C. Hsueh, T. K. Tsai und R. K. Iyer, „Fault injection techniques and tools“, *IEEE Computer*, Jg. 30, Nr. 4, S. 75–82, 1997. DOI: 10.1109/2.585157
- [11] Django Software Foundation, *Databases – Persistent connections*, Django Documentation. [Online]. Available: <https://docs.djangoproject.com/en/stable/ref/databases/#persistent-connections>, Zuletzt abgerufen: 30. Mai 2026.
- [12] Python Software Foundation, *multiprocessing.shared_memory – Shared memory for direct access across processes*, Python 3 Documentation. [Online]. Available: https://docs.python.org/3/library/multiprocessing.shared_memory.html, Zuletzt abgerufen: 30. Mai 2026.

Eidesstattliche Erklärung

Hiermit versichere ich, die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt sowie die Zitate deutlich kenntlich gemacht zu haben.

Ich erkläre weiterhin, dass die vorliegende Arbeit in gleicher oder ähnlicher Form noch nicht im Rahmen eines anderen Prüfungsverfahrens eingereicht wurde.

Bad Hersfeld, den 1. Juni 2026

Luca Michael Schmidt